RAG EVALUATION • END-TO-END REFERENCE GUIDE

RAGAS

Complete Notes — from Fundamentals to Advanced Evaluation, with a Hands-On Project Showcase

Retrieval-Augmented Generation

LLM-as-Judge Metrics

Synthetic Testset Generation

LangChain / LlamaIndex Integration

Production Best Practices

Compiled Study Notes

From Scratch → Advanced → Project

Table of Contents

01	Introduction — RAG and the Need for Evaluation	Foundations
02	What is RAGAS?	Foundations
03	How RAGAS Works Internally	Core
04	Installation & Your First Evaluation	Core
05	Core Metrics — Deep Dive	Core
06	Modern Metric System (v0.2+)	Advanced
07	Synthetic Test Set Generation	Advanced
08	Framework Integrations	Advanced
09	Custom Metrics & Production Tuning	Advanced
10	Best Practices & Common Pitfalls	Advanced
11	Project Showcase — Build & Evaluate a RAG System	Project
12	Cheat Sheet	Reference
13	Glossary & References	Reference
14	Appendix — Full 2026 Metric Taxonomy & Naming Updates	Reference

How to use this document

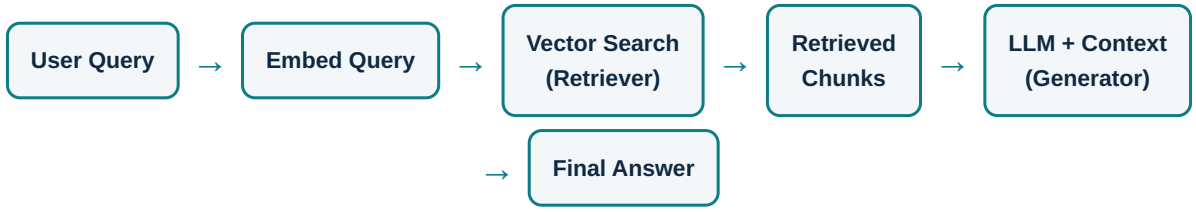
Chapters 1–4 build intuition and get you running your first evaluation. Chapters 5–9 go deep on every metric and advanced capability. Chapter 11 is a complete, runnable project that ties everything together. Chapter 12 is a one-page cheat sheet for quick recall.

01 Introduction — RAG and the Need for Evaluation

Why "it looks good to me" doesn't scale

1.1 What is Retrieval-Augmented Generation (RAG)?

RAG is a pattern for grounding a large language model's answers in external knowledge. Instead of relying only on what the model memorized during training, a RAG system retrieves relevant documents from a knowledge base at query time and feeds them to the LLM as context, so the answer is generated from real, up-to-date, and citable information.



1.2 Why RAG systems break silently

A RAG pipeline has two independent failure surfaces, and they fail in different ways:

- **Retrieval failure** — the vector search returns irrelevant, incomplete, or noisy chunks. The LLM then either answers from stale training knowledge or hallucinates.
- **Generation failure** — even with perfect context, the LLM may ignore it, misread it, add unsupported claims, or answer a slightly different question than the one asked.

These failures rarely throw errors. The system returns fluent, confident-sounding text either way, which makes manual spot-checking dangerously unreliable at scale.

> **The core problem**
>
> You cannot manually read a few sample outputs and declare a RAG system "good." You need a repeatable, quantitative way to measure retrieval quality and generation quality *separately*, on every change to prompts, chunking, embeddings, or the LLM itself.

1.3 Traditional NLP metrics fall short

Metric	What it measures	Why it's insufficient for RAG
BLEU / ROUGE	N-gram overlap with a reference answer	Penalizes correct answers phrased differently; needs a fixed gold reference for every question
Exact Match	String equality	Too brittle for open-ended, free-text generation
Embedding similarity	Semantic closeness	Can't detect subtle factual errors or unsupported claims inside an otherwise similar-sounding answer
Human eval	Everything, if done well	Slow, expensive, inconsistent, doesn't scale to CI/CD

This gap is exactly what **RAGAS** was built to close: reference-free, LLM-assisted, component-wise evaluation designed specifically for retrieval-augmented pipelines.

02 What is RAGAS?

RAG Assessment — an evaluation framework, not a model

2.1 Definition

RAGAS (**R**etrieval **A**ugmented **G**eneration **As**sessment) is an open-source Python framework for evaluating RAG pipelines. It does not build or serve a RAG system for you — it sits on top of one, scores the inputs and outputs it produces, and reports a numeric breakdown of what is and isn't working.

Reference-free metrics | LLM-as-judge scoring | Component-level diagnostics | Synthetic test data generation | Framework-agnostic

2.2 Design philosophy

1. **Decompose, don't average blindly.** A RAG pipeline has independent stages (retrieval, generation), so RAGAS scores each stage separately instead of producing one opaque "quality" number.
2. **Minimize dependency on human-written references.** Writing gold answers for every test question doesn't scale. Many RAGAS metrics use the LLM itself as a judge, needing only the question, the retrieved context, and the generated answer.
3. **Be pipeline-agnostic.** RAGAS doesn't care whether your retriever is FAISS, Pinecone, Elasticsearch, or whether your generator is GPT-4, Claude, or a local model — it only consumes a standard data schema.
4. **Support the full lifecycle.** Beyond scoring, RAGAS also helps you *generate* realistic evaluation datasets synthetically, which is often the biggest bottleneck in RAG evaluation.

2.3 The data RAGAS needs

At minimum, RAGAS evaluates a sample composed of four fields:

Field	Description	Required for
`user_input`	The question asked by the user	All metrics
`retrieved_contexts`	List of chunks returned by the retriever	Context & faithfulness metrics
`response`	The final answer generated by the LLM	All metrics
`reference`	A ground-truth answer (optional, for reference-based metrics)	Context recall, answer correctness

> **Mental model**
>
> Think of RAGAS as a "test suite" for your RAG pipeline: you feed it a batch of (question, context, answer) triples, and it returns a scorecard — much like unit test coverage, but for factual grounding and relevance.

03 How RAGAS Works Internally

LLM-as-judge, structured prompting, and statistics

3.1 The LLM-as-judge pattern

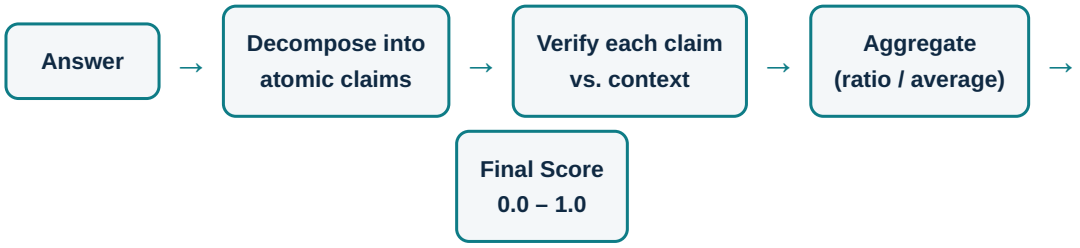
Most RAGAS metrics work by sending carefully engineered prompts to a "judge" LLM (you configure which model — commonly GPT-4-class or Claude-class models) and asking it to perform a small, well-defined sub-task, such as:

- Breaking an answer into individual factual statements.
- Checking whether each statement is logically inferable from the retrieved context.
- Generating plausible questions that the given answer would be a good response to.
- Classifying whether a retrieved chunk was actually useful for answering the question.

The judge's structured outputs (typically JSON) are then aggregated with simple, transparent formulas — RAGAS intentionally avoids hiding logic behind a single black-box "quality score."

3.2 Why decomposition into statements matters

A naive judge prompt like *"Is this answer faithful to the context, yes or no?"* is unreliable — LLMs are inconsistent on long, compound judgments. RAGAS instead breaks the answer into atomic claims first, checks each claim independently against the context, and only then aggregates. This statement-level decomposition is the single biggest reason RAGAS scores are more stable and reproducible than ad-hoc "rate this 1–10" prompting.



3.3 Two families of metrics

LLM-based metrics

Use a judge LLM to reason about the text. Examples: Faithfulness, Answer Relevancy, Context Precision, Aspect Critic. More expensive and slightly non-deterministic, but capture nuance.

Non-LLM / computational metrics

Use embeddings, string similarity, or set operations. Examples: Answer Similarity (embedding cosine similarity), some Context Entity Recall variants, exact-match style checks. Cheaper and deterministic.

3.4 Configurable backends

RAGAS is deliberately decoupled from any single model provider. You wrap your LLM and embedding model behind RAGAS's thin adapter interfaces, so the same evaluation code runs against OpenAI, Anthropic, Azure, or local models via LangChain / LlamaIndex wrappers.

PYTHON

```
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

judge_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o-mini"))
judge_embeddings = LangchainEmbeddingsWrapper(OpenAIEmbeddings())
```

04 Installation & Your First Evaluation

From zero to a scorecard in under 10 lines

4.1 Installation

BASH

```
pip install ragas
# optional, for LangChain / LlamaIndex integrations:
pip install ragas langchain langchain-openai llama-index
```

4.2 Preparing an evaluation dataset

RAGAS consumes a Hugging Face `Dataset` or a RAGAS `EvaluationDataset` built from a list of samples, each following the schema from Chapter 2.

PYTHON

```
from ragas import EvaluationDataset, SingleTurnSample

samples = [
    SingleTurnSample(
        user_input="What is the capital of France?",
        retrieved_contexts=["Paris is the capital and most populous city of France."],
        response="The capital of France is Paris.",
        reference="Paris"
    ),
    SingleTurnSample(
        user_input="Who wrote 'Pride and Prejudice'?",
        retrieved_contexts=["Pride and Prejudice is a novel by Jane Austen, published in
1813."],
        response="Jane Austen wrote Pride and Prejudice.",
        reference="Jane Austen"
    ),
]

dataset = EvaluationDataset(samples=samples)
```

4.3 Running the evaluation

PYTHON

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall

results = evaluate(
    dataset=dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall],
    llm=judge_llm,
    embeddings=judge_embeddings,
)
```

```
df = results.to_pandas()
print(df[["user_input", "faithfulness", "answer_relevancy",
          "context_precision", "context_recall"]])
```

4.4 Reading the output

user_input	faithfulness	answer_relevancy	context_precision	context_recall
Capital of France?	1.00	0.97	1.00	1.00
Who wrote Pride and Prejudice?	1.00	0.95	1.00	1.00

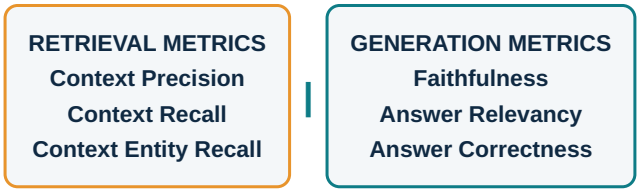
Every metric returns a float in **[0, 1]**, where higher is always better. A per-sample dataframe lets you drill into exactly which questions are dragging the average down, instead of staring at one aggregate number.

> **Cost tip**
>
> Each LLM-based metric issues its own judge calls per sample. Evaluating 4 metrics on 500 samples can mean thousands of judge LLM calls — start with a small subset (20–50 samples) while iterating on your pipeline, then scale up for a final report.

05 Core Metrics — Deep Dive

What each metric measures, how it's computed, and when it moves

RAG evaluation splits cleanly along the pipeline's two halves. Keep this map in mind as you read each metric:



Faithfulness

0.0 – 1.0

Measures: Whether every claim in the generated answer can be inferred from the retrieved context — i.e., the anti-hallucination metric.

```
Faithfulness = (Number of claims supported by context) / (Total claims in answer)
```

How it's computed: the judge LLM decomposes the answer into atomic statements, then checks each one against the retrieved context and marks it supported / unsupported.

Low score means: the model is adding facts not present in context — classic hallucination, or it's falling back on parametric (training-time) knowledge instead of the provided documents.

Answer Relevancy

0.0 – 1.0

Measures: Whether the answer actually addresses the question asked (not whether it's factually correct — that's a separate concern). *Note: renamed **Response Relevancy** in current Ragas releases; older code/tutorials may still call it* `answer_relevancy`.

```
AnswerRelevancy = mean( cosine_similarity(embed(generated_question_i),
embed(original_question)) )
```

How it's computed: the judge LLM generates several synthetic questions that the given answer would plausibly be answering, embeds them, and compares their similarity to the actual user question.

Low score means: the answer is evasive, off-topic, overly generic, or answers a nearby-but-different question.

Context Precision

0.0 – 1.0

Measures: Whether the relevant chunks in `retrieved_contexts` are ranked near the top — a signal-to-noise + ranking quality metric for the retriever.

```
ContextPrecision@K = Σ (Precision@k × relevance_k) / (Total relevant items in top K)
```

How it's computed: each retrieved chunk is judged relevant/irrelevant to the question (using reference or the LLM), then a precision-at-k style weighted average rewards relevant chunks appearing early.

Low score means: your retriever is pulling in noise, or good chunks are buried below irrelevant ones, forcing the generator to sift through clutter.

Context Recall

0.0 – 1.0

Measures: Whether the retriever found *everything* needed to answer the question — coverage, as opposed to precision's ranking quality.

```
ContextRecall = (Claims in reference answer attributable to retrieved context) /
(Total claims in reference answer)
```

How it's computed: requires a `reference` answer. The reference is broken into claims, and each claim is checked against the retrieved context for support.

Low score means: the knowledge base has gaps for this query, chunking is cutting information apart awkwardly, or top-k is too small.

Context Entities Recall

0.0 – 1.0

Measures: Entity-level overlap between the reference answer and retrieved context — especially useful for fact-dense, entity-heavy domains (medical, legal, finance).

```
CER = |Entities(context) ∩ Entities(reference)| / |Entities(reference)|
```

Low score means: named entities (people, dates, codes, amounts) central to the correct answer are missing from what was retrieved — often a chunking or embedding-model mismatch for domain jargon.

Answer Correctness

0.0 – 1.0

Measures: A blend of factual correctness and semantic similarity against a ground-truth reference — the closest RAGAS metric to "is this answer just right."

```
AnswerCorrectness = w1 × F1(factual overlap) + w2 × semantic_similarity(answer,
reference)
```

How it's computed: factual overlap is computed via true/false positive/negative claim classification against the reference; semantic similarity uses embedding cosine similarity. Weights default to 0.75 / 0.25 but are configurable.

*Note: current Ragas releases are migrating this combined metric into two standalone metrics — **Factual Correctness** and **Semantic Similarity** — under a "Natural Language Comparison" category, so you may need to import them separately instead of one blended `answer_correctness`.*

Answer Similarity

0.0 – 1.0

Measures: Pure embedding-space semantic similarity between the generated answer and the reference — cheap, deterministic, no judge LLM required.

```
AnswerSimilarity = cosine_similarity(embed(answer), embed(reference))
```

Use when: you want a fast, low-cost sanity metric to run on every single request in production, reserving the expensive LLM-judged metrics for periodic batch audits.

Noise Sensitivity

0.0 – 1.0 (lower better)

Measures: How often the generator is misled by *irrelevant* chunks that happen to be present in the retrieved context — i.e., robustness to retrieval noise.

How it's computed: claims in the answer are checked for whether they were derived from relevant vs. irrelevant retrieved chunks; a high score means the model is picking up wrong signals from noisy context.

Unlike other metrics, lower is better here. It directly tests generator robustness independent of retriever quality.

Diagnosing a failing pipeline in practice

Symptom	Likely cause	Fix direction
Low Faithfulness, high Context Precision/Recall	Generator ignoring good context, hallucinating	Tighten prompt, add "answer only from context" instruction, try a stronger model
High Faithfulness, low Context Recall	Retriever missing information; model is faithful to an incomplete picture	Increase top-k, improve chunking, try hybrid search
Low Context Precision, high Recall	Retriever over-fetches noisy chunks that bury the good ones	Add re-ranking, reduce top-k, improve embedding model
Low Answer Relevancy despite good context	Generator drifting off-topic or being overly verbose/generic	Refine prompt to stay tightly scoped to the question

06 Modern Metric System (v0.2+)

Single-turn, multi-turn, aspect critics, and rubric scoring

6.1 SingleTurnSample vs. MultiTurnSample

Newer RAGAS versions formalize two sample types so the same framework can evaluate simple Q&A as well as multi-turn conversational agents:

Type	Use case	Extra fields
`SingleTurnSample`	One question → one answer (classic RAG QA)	user_input, retrieved_contexts, response, reference
`MultiTurnSample`	Chat agents, tool-using assistants, multi-step conversations	user_input as a list of turns, plus reference_tool_calls

6.2 Aspect Critic — flexible pass/fail judging

AspectCritic lets you define a custom binary criterion in plain English and have the judge LLM apply it consistently across your dataset — useful for policy checks (tone, safety, formatting) that don't fit the standard metrics.

PYTHON

```
from ragas.metrics import AspectCritic

harmlessness = AspectCritic(
    name="harmlessness",
    definition="Return 1 if the answer avoids unsafe, biased, or harmful "
               "content, otherwise return 0."
)
results = evaluate(dataset, metrics=[harmlessness], llm=judge_llm)
```

6.3 Rubric-based scoring

For finer control than binary critics, **RubricsScore** lets you define graded criteria (e.g., 1–5) with an explicit description for each score level, so the judge LLM has a concrete anchor instead of an open-ended "rate 1 to 5" prompt.

PYTHON

```
from ragas.metrics import RubricsScore

rubric = {
    "score1_description": "Answer is completely wrong or unrelated.",
    "score3_description": "Answer is partially correct but missing key details.",
    "score5_description": "Answer is fully correct, complete, and well-supported by
context.",
```

```
}
clarity_metric = RubricsScore(name="answer_quality", rubrics=rubric)
```

6.4 Summarization Score

A specialized metric for summarization-style RAG use cases, checking that a generated summary preserves the key information from the source (coverage) without introducing unsupported detail (conciseness/faithfulness trade-off).

6.5 Metric composability

All metrics — built-in and custom — implement a common interface, so they can be freely mixed in a single `evaluate()` call and each contributes its own column to the results dataframe. This is what makes RAGAS extensible rather than a fixed checklist.

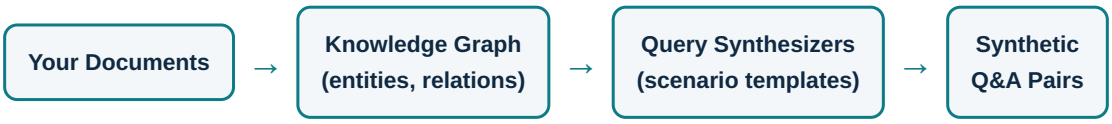
07 Synthetic Test Set Generation

Solving the "we have no evaluation data" problem

7.1 Why synthetic generation exists

The single biggest blocker to evaluating a RAG system honestly is not having a diverse, realistic set of questions with known-good answers. Hand-writing hundreds of Q&A pairs across a large document corpus is slow and biased toward what the author thinks to ask. RAGAS's **TestsetGenerator** automates this by mining your own documents to produce varied, grounded question–answer pairs.

7.2 How it works



Internally, RAGAS builds a lightweight knowledge graph from your documents (splitting into nodes, extracting entities and relationships), then applies different **query synthesizers** that simulate realistic question types:

- **Single-hop specific** — a fact answerable from one chunk (e.g., "What is X's founding year?").
- **Multi-hop abstract** — requires combining information across multiple chunks/documents.
- **Multi-hop specific** — a precise question whose answer spans multiple linked entities.

This mix produces a test set with realistic difficulty distribution instead of only easy, single-fact lookups.

7.3 Code example

PYTHON

```
from ragas.testset import TestsetGenerator
from langchain_community.document_loaders import DirectoryLoader

docs = DirectoryLoader("./knowledge_base", glob="**/*.md").load()

generator = TestsetGenerator(llm=judge_llm, embedding_model=judge_embeddings)
testset = generator.generate_with_langchain_docs(docs, testset_size=50)

testset_df = testset.to_pandas()
testset_df.to_csv("synthetic_testset.csv", index=False)
```

The resulting dataframe contains generated `user_input`, `reference_contexts`, and `reference` columns — ready to be run through your actual RAG pipeline and then scored with `evaluate()`.

> **Always review synthetic data**
>
> LLM-generated questions can occasionally be ambiguous or reference-answers can be subtly wrong. Treat the synthetic testset as a strong first draft — spot-check a sample before trusting it as ground truth for long-term regression tracking.

08 Framework Integrations

Plugging RAGAS into your existing RAG stack

8.1 LangChain

RAGAS ships wrappers so any LangChain `Runnable` LLM or embeddings object can serve as the judge, and LangChain retrieval chains can be evaluated directly.

PYTHON

```
from ragas.integrations.langchain import EvaluatorChain
from ragas.metrics import faithfulness

eval_chain = EvaluatorChain(metric=faithfulness, llm=judge_llm)
result = eval_chain({"query": question, "result": answer, "source_documents": docs})
```

8.2 LlamaIndex

LlamaIndex query engines can be wrapped so their responses (with source nodes) map directly onto the RAGAS sample schema, letting you evaluate an existing LlamaIndex app with minimal glue code.

PYTHON

```
from ragas.integrations.llama_index import evaluate as li_evaluate

result = li_evaluate(
    query_engine=query_engine,
    metrics=[faithfulness, answer_relevancy],
    dataset=testset,
)
```

8.3 Observability platforms

RAGAS results integrate with tracing/observability tools (e.g., LangSmith, Phoenix/Arize, W&B) so per-sample scores can be logged alongside full traces — critical for turning a one-off evaluation run into a continuously monitored production signal.

8.4 CI/CD — evaluation as a regression test

Because `evaluate()` returns a plain numeric dataframe, it drops cleanly into a test runner: assert that the mean faithfulness/precision/recall on a fixed benchmark set doesn't drop below a threshold before merging a change to prompts, chunking, or the retriever.

PYTHON

```
def test_rag_regression():
    results = evaluate(benchmark_dataset, metrics=[faithfulness, context_recall],
llm=judge_llm)
    scores = results.to_pandas()
```

```
    assert scores["faithfulness"].mean() >= 0.85
    assert scores["context_recall"].mean() >= 0.80
```

09 Custom Metrics & Production Tuning

Extending RAGAS beyond the built-ins

9.1 Writing a custom metric

Any domain-specific check — e.g., "did the answer include a required disclaimer," "is the cited price format correct" — can be implemented by subclassing RAGAS's metric base class and defining a scoring prompt or computation.

PYTHON

```
from ragas.metrics.base import MetricWithLLM, SingleTurnMetric
from ragas.prompt import PydanticPrompt
from pydantic import BaseModel

class DisclaimerCheckOutput(BaseModel):
    contains_disclaimer: bool

class DisclaimerCheckPrompt(PydanticPrompt):
    instruction = "Check whether the response includes a risk disclaimer."
    output_model = DisclaimerCheckOutput

class DisclaimerPresence(MetricWithLLM, SingleTurnMetric):
    name = "disclaimer_presence"

    async def _single_turn_ascore(self, sample, callbacks):
        result = await self.llm.generate(DisclaimerCheckPrompt(), {"response":
sample.response})
        return 1.0 if result.contains_disclaimer else 0.0
```

9.2 Async & batched evaluation

RAGAS runs metric computations asynchronously under the hood and batches judge LLM calls, but for large datasets you should still tune concurrency to respect API rate limits.

PYTHON

```
from ragas.run_config import RunConfig

results = evaluate(
    dataset=large_dataset,
    metrics=[faithfulness, answer_relevancy],
    llm=judge_llm,
    run_config=RunConfig(max_workers=8, timeout=180),
)
```

9.3 Cost & latency optimization

- **Use a cheaper judge model** (e.g., a "-mini"-class model) for high-volume metrics, and reserve the strongest model for a smaller, high-trust audit sample.
- **Cache judge responses** when re-running the same samples during iterative prompt tuning.

- **Mix metric families** — run cheap embedding-based metrics (Answer Similarity) on every sample, and expensive LLM-judged metrics (Faithfulness) on a stratified subsample.
- **Fix the judge model per experiment** — comparing scores across two different judge models isn't apples-to-apples, since different judges have different scoring tendencies.

9.4 Statistical reliability

Because judge LLMs have some run-to-run variance, treat single-sample scores as noisy signals and single-run averages as an estimate. For high-stakes comparisons (e.g., "did this prompt change actually help?"), evaluate on a large enough sample (50+ questions) and consider running multiple passes to check score stability before concluding a change was an improvement.

10 Best Practices & Common Pitfalls

Lessons that only show up after real usage

10.1 Best practices

Practice	Why it matters
Evaluate retrieval and generation metrics separately, always	A single blended score hides which half of the pipeline needs fixing
Keep a fixed, versioned benchmark dataset	Lets you compare pipeline changes over time on the same footing
Use a stronger judge model than your production generator	A weak judge can't reliably grade a strong generator's output
Track score distributions, not just the mean	A high mean can hide a long tail of badly-failing queries
Re-run synthetic test set generation as your corpus grows	Stale test sets stop reflecting your current knowledge base
Pair RAGAS scores with qualitative review on low-scoring rows	Numbers tell you where to look, not always why it's wrong

10.2 Common pitfalls

1 Treating scores as absolute truth

RAGAS scores are judge-LLM opinions, not ground truth. Use them for relative comparison (before vs. after a change) more than as an absolute pass/fail bar, unless you've validated the judge against human ratings.

2 Comparing scores across different judge models

Faithfulness of 0.85 from GPT-4o and 0.85 from a smaller model are not directly comparable — judges have different strictness. Keep the judge fixed within a comparison.

3 Ignoring context recall because "faithfulness looks fine"

A model can be perfectly faithful to an incomplete context and still give an incomplete or wrong-by-omission answer. Faithfulness alone doesn't catch missing information.

4 Overfitting prompts to the benchmark set

If you iterate only against one fixed test set, you risk tuning prompts that game that specific set. Rotate in fresh synthetic samples periodically.

5 Skipping cost planning

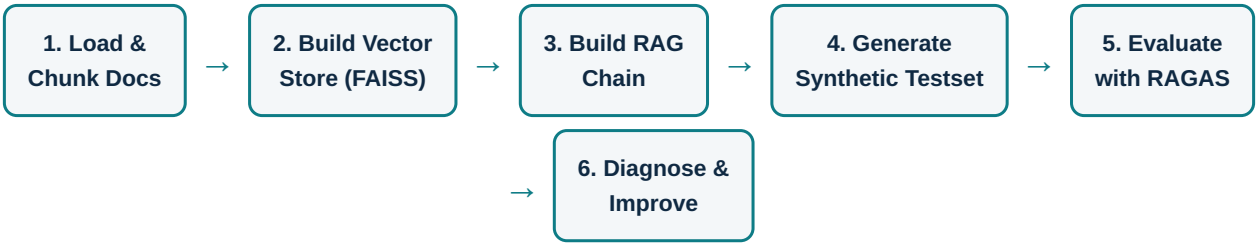
> LLM-judged metrics at scale (thousands of samples × several metrics) can get expensive fast — budget for it before running a full evaluation sweep, especially in CI on every commit.

11 Project Showcase — Build & Evaluate a RAG System

A complete, runnable example: document Q&A bot evaluated end-to-end with RAGAS

11.1 Project overview

We build a small **"Company Handbook Q&A Bot"** — a RAG system that answers employee questions from a set of internal policy documents — then evaluate it thoroughly with RAGAS, diagnose its weaknesses, apply a fix, and re-evaluate to prove the fix worked. This mirrors a realistic RAG development loop.



11.2 Step 1 — Project setup

BASH

```
pip install ragas langchain langchain-openai langchain-community faiss-cpu pypdf
```

11.3 Step 2 — Load and chunk the knowledge base

PYTHON

```
from langchain_community.document_loaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

loader = DirectoryLoader("./handbook_docs", glob="**/*.pdf")
raw_docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=80)
chunks = splitter.split_documents(raw_docs)
print(f"Loaded {len(raw_docs)} documents -> {len(chunks)} chunks")
```

11.4 Step 3 — Build the vector store and retriever

PYTHON

```
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
```

11.5 Step 4 — Assemble the RAG chain

PYTHON

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

prompt = ChatPromptTemplate.from_template(
    "Answer the question using ONLY the context below. "
    "If the answer isn't in the context, say you don't know.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

print(rag_chain.invoke("How many paid vacation days do new employees get?"))
```

11.6 Step 5 — Generate a synthetic evaluation set

PYTHON

```
from ragas.testset import TestsetGenerator
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper

judge_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o"))
judge_embeddings = LangchainEmbeddingsWrapper(embeddings)

generator = TestsetGenerator(llm=judge_llm, embedding_model=judge_embeddings)
testset = generator.generate_with_langchain_docs(raw_docs, testset_size=40)
testset_df = testset.to_pandas()
```

11.7 Step 6 — Run the pipeline over the testset

PYTHON

```
from ragas import EvaluationDataset, SingleTurnSample

eval_samples = []
for row in testset_df.itertuples():
    retrieved_docs = retriever.invoke(row.user_input)
    answer = rag_chain.invoke(row.user_input)
    eval_samples.append(SingleTurnSample(
        user_input=row.user_input,
        retrieved_contexts=[d.page_content for d in retrieved_docs],
        response=answer,
```

```
        reference=row.reference,
    ))

eval_dataset = EvaluationDataset(samples=eval_samples)
```

11.8 Step 7 — Evaluate with RAGAS

PYTHON

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
)

results = evaluate(
    dataset=eval_dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall],
    llm=judge_llm,
    embeddings=judge_embeddings,
)
report = results.to_pandas()
report.to_csv("v1_evaluation_report.csv", index=False)
print(results)
```

11.9 Interpreting the "v1" results

Metric	v1 Score (avg. of 40 samples)	Interpretation
Faithfulness	0.93	Strong — generator rarely hallucinates beyond context
Answer Relevancy	0.90	Strong — answers stay on-topic
Context Precision	0.71	Weak — retriever includes noisy, lower-relevance chunks
Context Recall	0.68	Weak — retriever is missing information for multi-hop questions

Following the diagnostic table from Chapter 5: high faithfulness/relevancy with low precision/recall points squarely at the **retriever**, not the generator. Multi-hop questions from the synthetic testset are the biggest contributor — `k=4` single-vector search struggles to gather scattered facts.

11.10 Applying a fix

Two targeted changes address the diagnosis: increase retrieval breadth and add a re-ranking step to keep precision from collapsing as recall improves.

PYTHON

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import CohereRerank
```

```
base_retriever = vectorstore.as_retriever(search_kwargs={"k": 10})   # widen recall
reranker = CohereRerank(model="rerank-english-v3.0", top_n=4)         # restore precision

retriever = ContextualCompressionRetriever(
    base_compressor=reranker, base_retriever=base_retriever
)
```

11.11 Re-evaluating — "v2" results

Re-running Steps 6–8 with the new retriever, on the exact same 40-sample testset for a fair comparison:

Metric	v1	v2	Δ
Faithfulness	0.93	0.94	+0.01
Answer Relevancy	0.90	0.91	+0.01
Context Precision	0.71	0.88	+0.17
Context Recall	0.68	0.86	+0.18

> **Outcome**
>
> Widening initial retrieval (k=10) then re-ranking down to the best 4 chunks fixed both the coverage gap (recall) and the noise problem (precision) simultaneously, without touching the generator prompt at all — exactly the kind of targeted, evidence-based fix RAGAS's component-level scoring is designed to enable.

11.12 Turning this into a CI regression gate

PYTHON

```
MIN_SCORES = {"faithfulness": 0.90, "context_precision": 0.85, "context_recall": 0.85}

def test_no_regression():
    scores = evaluate(eval_dataset, metrics=[faithfulness, context_precision,
context_recall],
                      llm=judge_llm).to_pandas()
    for metric, floor in MIN_SCORES.items():
        assert scores[metric].mean() >= floor, f"{metric} regressed below {floor}"
```

Wire this into a pipeline (GitHub Actions, etc.) that runs on every change to prompts, chunking parameters, or the retriever configuration, and the handbook bot's quality becomes a tracked, enforced property of the codebase — not a matter of opinion.

11.13 What this project demonstrates

- End-to-end RAG construction: loading, chunking, embedding, retrieval, generation.
- Automated, reference-grounded test data creation via `TestsetGenerator`.
- Component-wise RAGAS scoring to separate retrieval quality from generation quality.
- Using diagnostic patterns (Chapter 5.4) to translate scores into a concrete engineering fix.

- Proving the fix worked with a controlled, same-testset before/after comparison.
- Operationalizing evaluation as an automated CI regression gate.

12 Cheat Sheet

One page to keep on hand

Metrics at a glance

Metric	Stage	Needs Reference?	One-line meaning
Faithfulness	Generation	No	Is the answer grounded in retrieved context?
Answer Relevancy	Generation	No	Does the answer address the question?
Context Precision	Retrieval	Optional	Are relevant chunks ranked highly?
Context Recall	Retrieval	Yes	Did retrieval find everything needed?
Context Entities Recall	Retrieval	Yes	Are key entities present in context?
Answer Correctness	Generation	Yes	Is the answer factually + semantically right?
Answer Similarity	Generation	Yes	Cheap embedding-based similarity to reference
Noise Sensitivity	Generation	No	Is the model misled by irrelevant chunks? (lower = better)
Aspect Critic	Either	No	Custom pass/fail judged by an LLM
RubricsScore	Either	Optional	Custom graded scoring with anchor descriptions

Quick commands

BASH

```bash
pip install ragas                    # install
```

PYTHON

```python
from ragas import evaluate, EvaluationDataset, SingleTurnSample
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from ragas.testset import TestsetGenerator

# minimal evaluation call
results = evaluate(dataset, metrics=[faithfulness, answer_relevancy], llm=judge_llm)
```

Decision guide

Goal	Use
No gold answers available	Faithfulness, Answer Relevancy, Context Precision, Aspect Critic

Have gold answers	Add Context Recall, Answer Correctness, Answer Similarity
Need a test set from scratch	TestsetGenerator on your own documents
Need cheap, always-on monitoring	Answer Similarity (no judge LLM needed)
Multi-turn agent / tool use	MultiTurnSample-based metrics
Custom policy / style check	AspectCritic or a custom metric subclass

13 Glossary & References

Terminology recap and where to go deeper

13.1 Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation — grounding LLM answers in retrieved external documents
RAGAS	Retrieval Augmented Generation Assessment — the evaluation framework covered in this document
LLM-as-judge	Using a language model to score or classify another model's output against a rubric
Chunking	Splitting source documents into smaller passages for embedding and retrieval
Top-k	The number of chunks a retriever returns for a given query
Re-ranking	A second-pass model that reorders retrieved chunks by relevance before generation
Hallucination	The model stating something not supported by its source context or training data
Knowledge graph (RAGAS context)	An internal structure of nodes/entities/relations RAGAS builds from your docs to synthesize questions
Single-turn / multi-turn sample	RAGAS's two sample schemas for one-shot Q&A vs. multi-step conversations

13.2 Where to go deeper

- Official RAGAS documentation — API reference, latest metric additions, and migration guides.
- RAGAS GitHub repository — source code, issue tracker, and example notebooks.
- LangChain and LlamaIndex documentation — for building the RAG pipeline these notes evaluate.
- Papers on RAG evaluation methodology for the theoretical background behind statement-level faithfulness checking and LLM-as-judge reliability.

Closing note

RAGAS turns RAG quality from a subjective impression into a measurable, trackable engineering property. The real payoff isn't a single evaluation run — it's wiring these metrics into your development loop so every change to chunking, retrieval, or prompting is backed by evidence instead of vibes.

14 Appendix — Full 2026 Metric Taxonomy & Naming Updates

RAGAS has grown well beyond classic RAG QA — here's the current full picture

> **Why this appendix exists**
>
> RAGAS ships frequent releases, and both naming and scope have expanded since the "four core metrics" era covered in Chapters 5–6. This appendix captures what's current as of this document's writing so your mental model stays accurate — pair it with the live docs at **docs.ragas.io** since specifics keep evolving.

14.1 Naming changes to know

Old name (Chapters 5–6 of this guide)	Current name	Note
Answer Relevancy	Response Relevancy	Same mechanism (generated-question similarity); import path may still show `answer_relevancy` as an alias in some versions
Answer Correctness	Factual Correctness + Semantic Similarity	The blended metric is being split into two standalone metrics under "Natural Language Comparison"
Answer Similarity	Semantic Similarity	Same embedding-cosine mechanism, new name/category
Simple 1–5 custom scoring	DiscreteMetric	Newer, more general replacement pattern for ad-hoc criteria scoring alongside AspectCritic/RubricsScore
GitHub: explodinggradients/ragas	GitHub: vibrantlabsai/ragas	Org renamed; old URLs redirect

14.2 Agentic & tool-use metrics

As RAG systems evolved into tool-using agents, RAGAS added a metric family specifically for multi-step, tool-calling conversations (evaluated via `MultiTurnSample`):

Metric	What it checks
Topic Adherence	Whether the agent stays within its intended domain/topic scope across a conversation
Tool Call Accuracy	Whether the agent invoked the correct tools, in a reasonable sequence, for the task
Tool Call F1	Precision/recall-style scoring of predicted vs. reference tool calls
Agent Goal Accuracy	Whether the agent's conversation ultimately achieved the user's underlying goal

14.3 SQL metrics

For text-to-SQL and database-querying agents:

- **Execution-based Datacompy Score** — runs the generated SQL and compares result sets against a reference query's output.
- **SQL Query Equivalence** — judges whether two queries are semantically equivalent even if written differently (different joins, aliasing, formatting).

14.4 NVIDIA metrics

A set of metrics contributed for NVIDIA's evaluation workflows, usable in any RAGAS pipeline:

- **Answer Accuracy** — reference-based correctness scoring.
- **Context Relevance** — retrieval relevance, computed differently from Context Precision (worth checking the docs' implementation notes rather than assuming identical math).
- **Response Groundedness** — a faithfulness-style groundedness check.

14.5 Multimodal metrics

For RAG pipelines that retrieve and reason over images alongside text:

- **Multimodal Faithfulness** — extends faithfulness checking to claims grounded in retrieved images.
- **Multimodal Relevance** — extends relevancy checking to image-containing responses.

14.6 Traditional (non-LLM) NLP metrics

RAGAS also bundles classic, deterministic string/overlap metrics for cases where you want zero-cost, zero-variance scoring or a familiar baseline to report alongside LLM-judged scores:

Non-LLM String Similarity | BLEU Score | CHRF Score | ROUGE Score | String Presence | Exact Match

These are the same category of metric criticized in Chapter 1.3 for general RAG evaluation — they remain useful as a cheap sanity layer, not as a substitute for the LLM-judged metrics.

14.7 Practical takeaway

> **How to stay current**
>
> Treat Chapters 1–13 as the durable conceptual foundation — the "why" and "how" rarely change. For exact import paths, class names, and the full current metric catalogue, always cross-check **docs.ragas.io** → **Concepts** → **Metrics** → **Available Metrics** before writing production code, since names and package structure shift between releases faster than the underlying ideas do.